UNIDAD 9
GESTIÓN DE LA SEGURIDAD DE LOS DATOS

FICHA TÉCNICA DE LA UNIDAD DIDÁCTICA

Unidad Didáctica	Unidad Didáctica 12: Gestión de la Seguridad de los Datos
Resultados de Aprendizaje	**RA6:** Ejecuta tareas de aseguramiento de la información, analizándolas y aplicando mecanismos de salvaguarda y transferencia.
Criterios de Evaluación	CE6.a, CE6.b, CE6.c, CE6.d, CE6.e, CE6.f, CE6.g, CE6.h
Tiempo Estimado	6 Horas (Distribuidas en sesiones teórico-prácticas orientadas a la administración de sistemas)

ÍNDICE DE CONTENIDOS

1. INTRODUCCIÓN

La información corporativa constituye el activo más valioso de cualquier organización moderna. En el rol de un Administrador de Sistemas (SysAdmin), no basta con diseñar esquemas de datos relacionales eficientes y optimizar consultas complejas (competencias adquiridas en las Unidades 2, 3 y 5); es mandatorio garantizar de forma ininterrumpida la **disponibilidad**, **integridad** y **confidencialidad** de los datos frente a catástrofes físicas, fallos de software o accesos no autorizados.

Esta unidad didáctica se conecta de forma directa con los siguientes módulos del ciclo ASIR:

- **Seguridad y Alta Disponibilidad (SAD):** Donde se estudian políticas de seguridad perimetral, criptografía y redundancia a nivel de almacenamiento físico (RAID).
- **Administración de Sistemas Gestores de Bases de Datos (ASGBD - 2º curso):** En el cual se profundizará en la replicación maestro-esclavo, clustering y balanceo de carga.
- **Implantación de Sistemas Operativos (ISO):** Específicamente en la monitorización de servicios (daemons) del sistema y gestión de almacenamiento en volúmenes lógicos

(LVM).

2. DIAGNÓSTICO Y ORIGEN DE FALLOS EN UN SGBD

2.1 Tipologías de Fallo Comunes

Un SGBD puede sufrir interrupciones del servicio con pérdida potencial de consistencia o datos debido a múltiples factores que un SysAdmin debe prever de manera proactiva:

1. **Fallo de Suministro Eléctrico:** Cortes repentinos de energía que impiden la escritura de páginas de datos desde la caché en memoria (Buffer Pool) al disco duro físico.
2. **Fallos de Hardware:** Sectores defectuosos en unidades de disco duro, corrupción en los módulos de memoria RAM del servidor o fallos en la CPU.
3. **Fallos de Software:** Errores del sistema operativo (Kernel Panics), bloqueos del servicio mysqld o bugs en los controladores lógicos del sistema de archivos (e.g., ext4, XFS).
4. **Problemas del Sistema de Ficheros:** Agotamiento del espacio en disco o de la tabla de inodos, o corrupción interna en la estructura lógica de directorios de almacenamiento.
5. **Factores Externos o Catástrofes:** Incendios, inundaciones, robos o sabotajes físicos de la infraestructura del centro de datos.

La pérdida de datos en un SGBD puede categorizarse de dos formas:

- **Pérdida Parcial:** El sistema de archivos se mantiene intacto, pero se corrompen algunas páginas de datos correspondientes a tablas concretas, o se pierde la integridad referencial debido a transacciones incompletas.
- **Pérdida Total:** Destrucción total de los dispositivos de almacenamiento físico que contienen el directorio de datos o pérdida generalizada de la integridad semántica de todos los esquemas del servidor.

2.2 Elementos en un Plan de Recuperación de Desastres (DRP)

Para minimizar el impacto de estas caídas del sistema, se debe establecer una estrategia sistemática de mantenimiento preventivo y de recuperación:

- **Establecer un Plan de Respaldos Periódicos:** Automatizar copias de seguridad completas combinadas con copias incrementales o diferenciales, optimizando el espacio mediante técnicas de compresión y cifrado fuerte.

- ```
  Almacenamiento de Respaldos Offsite / Remotos:
  Conservar las copias en una ubicación geográfica distinta a la del
  servidor principal (e.g., almacenamiento en la nube o en servidores de
  respaldo remotos protegidos contra catástrofes).
  ```

- **Control del Acceso Físico y Lógico:** Protección de la sala de servidores frente a intrusiones físicas y despliegue de un sistema de privilegios de acceso lógico basado en el principio de mínimo privilegio (según el estándar DCL).

- **Planes de Contingencia Documentados y Testeados:** Realización de simulacros periódicos de recuperación de fallos para evaluar la viabilidad real de los planes de contingencia y medir los tiempos de recuperación (RTO - Recovery Time Objective) y los puntos de recuperación (RPO - Recovery Point Objective).

3. FICHEROS DE REGISTRO LOG DE MYSQL

3.1 Arquitectura de Almacenamiento Físico en Disco

En sistemas operativos Linux, el directorio por defecto donde MySQL almacena físicamente las bases de datos está configurado en la variable del sistema datadir:

```
# Comprobación de la ruta física desde la terminal del sistema
operativo
mysql -u root -p -e "SHOW VARIABLES LIKE 'datadir';"
```

Por defecto, la ruta es /var/lib/mysql. Dentro de este directorio, cada base de datos tiene asignado su propio subdirectorio con el nombre de la base de datos (por ejemplo, /var/lib/mysql/practica_dml).

Físicamente, los archivos se distribuyen de la siguiente manera según el motor de almacenamiento utilizado:
```

- **Archivos .frm:** Almacenan de manera exclusiva los metadatos de estructura de las tablas (válido para MyISAM e InnoDB en versiones previas a MySQL 8.0; en MySQL 8.0, esta información se consolida en el Diccionario de Datos del sistema).
- **Archivos .ibd:** Corresponden al motor por defecto **InnoDB**. Almacenan de forma agrupada los datos físicos y los índices asociados a cada tabla en modo file-per-table.
- **Archivos .myd:** Almacenan los datos en bruto de las tablas de tipo **MyISAM** (datos propiamente dichos).
- **Archivos .myi:** Almacenan los índices de las tablas creadas con el motor **MyISAM**.

3.2 Anatomía y Configuración de los Logs del Motor

MySQL proporciona cuatro logs esenciales para diagnosticar el estado del servidor y garantizar la recuperabilidad point-in-time:

1. **Log de Errores (log-error):** Registra eventos de arranque, parada y cualquier fallo crítico del servidor. Por defecto se nombra como host_name.err.
2. **Log de Consultas Lentas (slow_query_log):** Registra las sentencias cuya ejecución supera el umbral definido por long_query_time (en segundos). Es crucial para optimizar el rendimiento.
3. **Log General (general_log):** Registra de manera secuencial cada conexión abierta por un cliente y cada sentencia SQL recibida por el servidor. Consume una gran cantidad de I/O de disco y no se recomienda mantener activo de forma continua en producción.
4. **Log Binario (log_bin):** Contiene un registro cronológico de todas las sentencias que actualizan datos o alteran el esquema (DDL y DML de escritura). **Es indispensable para la replicación y para realizar restauraciones incrementales**. Almacenado en formato binario para optimizar la velocidad de escritura.

Para cerrar los archivos de log abiertos y forzar la inicialización de logs nuevos (muy útil para procesos de rotación y antes de copias de seguridad), se ejecuta la instrucción:

```
-- Forzar el volcado de buffers a disco y la rotación de logs
FLUSH LOGS;
```

🔗 **Documentación Oficial:** Para conocer más parámetros de los ficheros de log, lee MySQL Reference Manual - Server Logs.

3.3 La Utilidad mysqlbinlog

Dado que el registro binario se escribe en formato binario no legible, los SysAdmins deben utilizar la herramienta de terminal nativa **mysqlbinlog** para auditar o aplicar los cambios almacenados.

La sintaxis básica de ejecución desde el shell de Linux es:

```
mysqlbinlog [opciones] nombre_log_binario
```

Opciones Principales de mysqlbinlog

- --database=nombre_bd (-d): Filtra los eventos del log binario aplicando las sentencias que afecten de forma exclusiva al esquema indicado.
- --start-datetime="YYYY-MM-DD HH:MM:SS": Inicia la lectura de eventos desde la marca de tiempo absoluta especificada.
- --stop-datetime="YYYY-MM-DD HH:MM:SS": Detiene la lectura en la marca temporal provista (esencial para recuperar la base de datos hasta un instante inmediatamente anterior al desastre).
- --short-form (-s): Omite información de metadatos del binlog, imprimiendo únicamente las sentencias SQL puras.

Ejemplos Prácticos de mysqlbinlog sobre nuestra base de datos

```
# Ejemplo 1: Convertir un binlog a fichero de texto plano legible
para auditoría
mysqlbinlog --database=practica_dml /var/lib/mysql/binlog.000001 >
/tmp/auditoria_dml.sql
# Ejemplo 2: Aplicar de forma directa las instrucciones del binlog
para recuperar datos
mysqlbinlog --database=practica_dml /var/lib/mysql/binlog.000001 |
mysql -u root -p
```

🔗 **Documentación Oficial: Más Información en MySQL Manual - mysqlbinlog Utility**

4. MANTENIMIENTO Y REPARACIÓN DE TABLAS

Ante la sospecha de corrupción lógica de una tabla (por ejemplo, índices que no coinciden con los datos reales tras un apagón eléctrico), disponemos de utilidades de línea de comandos y sentencias SQL específicas.

4.1 Herramientas Externas: myisamchk y mysqlcheck

myisamchk (Exclusivo para tablas MyISAM)

Opera directamente a nivel de archivos físicos en disco (.MYD y .MYI). **Exige que el servidor MySQL esté completamente apagado o que las tablas implicadas estén bloqueadas contra escritura** para evitar la corrupción de datos.

```
# Comprobar estado de consistencia de la tabla fabricante (tipo
MyISAM si se requiere)
myisamchk --check /var/lib/mysql/practica_dml/fabricante.MYI
# Reparación estándar rápida reconstruyendo índices
myisamchk --recover --quick
/var/lib/mysql/practica_dml/fabricante.MYI
```

Opciones representativas: --silent (-s), --extended-check (-e), --force (-f), --backup (-B).

mysqlcheck (Motor genérico en caliente)

A diferencia de myisamchk, esta utilidad se comunica con el servidor MySQL en caliente (en ejecución) mediante comandos SQL nativos. Puede utilizarse para tablas InnoDB y MyISAM.

```
# Analizar y reparar automáticamente todas las tablas del esquema
'practica_dml'
mysqlcheck -u root -p --auto-repair --check practica_dml
```

🔗 **Documentación Oficial: Más Información en [MySQL Manual - mysqlcheck Tool]**

4.2 Sentencias SQL de Optimización e Integridad

Estas instrucciones se ejecutan directamente en un cliente interactivo o script SQL:

CHECK TABLE

Chequea la estructura de la tabla y valida sus índices asociados. Devolverá una tabla con el estado.

```
USE practica_dml;
CHECK TABLE empleado, producto;
```

ANALYZE TABLE

Analiza y recalcula las estadísticas de distribución de claves de los índices. Esto permite que el Optimizador de Consultas de MySQL elija planes de ejecución óptimos (uso de índices secundarios adecuados). Bloquea la tabla para lectura durante el procesamiento.

```
-- Analizar el estado de los índices en la tabla de empleados
ANALYZE TABLE empleado;
```

OPTIMIZE TABLE

Se utiliza para recuperar el espacio físico que ocupaban los registros previamente eliminados (mediante DELETE) en tablas con columnas de longitud variable (como VARCHAR o TEXT en la tabla fabricante de nuestro esquema practica_dml). Desfragmenta el archivo y reduce su tamaño en disco.

```
-- Desfragmentar la tabla fabricante tras realizar eliminaciones
masivas
OPTIMIZE TABLE fabricante;
```

REPAIR TABLE

Sintaxis SQL para intentar la reparación inmediata de tablas corruptas de tipo MyISAM (no compatible con InnoDB).

```
REPAIR TABLE fabricante QUICK;
```

🔗 **Documentación Oficial: Más Información en [MySQL Manual - Table Maintenance Statements].**

5. POLÍTICAS DE COPIAS DE SEGURIDAD

5.1 Clasificación Técnica de los Backups

Podemos realizar diferentes clasificaciones de las estrategias de respaldo:

- **En Frío (Cold Backup):** Requiere detener por completo el daemon mysqld. Es la forma más consistente de copiar archivos binarios directos, pero invalida la disponibilidad del servicio.
- **En Caliente (Hot Backup):** El servidor se mantiene activo sirviendo peticiones a aplicaciones cliente sin interrupción mientras se realiza el volcado de datos.
- **Físicas / Binarias:** Copia directa a nivel de bytes de los archivos contenidos en el datadir. Es un proceso extremadamente veloz tanto en el copiado como en la restauración.
- **Lógicas / Textuales:** Consiste en generar archivos con instrucciones de creación DDL y comandos de inserción DML (INSERT INTO). Son más lentas de procesar, pero altamente portables a otros SGBD o versiones de MySQL distintas.
- **Completas (Full Backup):** Copia íntegra de toda la estructura y datos del servidor.
- **Incrementales:** Copia de seguridad que almacena de forma única los cambios realizados en las tablas desde la última copia de seguridad realizada (completa o incremental). En MySQL se implementa copiando los logs binarios.

5.2 Copias Físicas o Binarias (InnoDB & MyISAM)

Para el motor por defecto de producción **InnoDB**, realizar una copia binaria en caliente requiere el uso de comandos avanzados para aislar los espacios de tablas.

Algoritmo de Respaldo Físico (Transporte de Tablespaces con practica_dml.producto)

Para clonar físicamente el archivo .ibd de una tabla InnoDB de forma consistente y restaurarlo en otra ubicación, el SysAdmin debe ejecutar de forma interactiva:

Paso 1: Solicitar un bloqueo de lectura en el servidor origen para garantizar que las páginas de datos en el Buffer Pool de la tabla producto se vuelquen de forma consistente a disco, cerrando la tabla temporalmente:

```
USE practica_dml;
FLUSH TABLES producto FOR EXPORT;
```

(Mantener la sesión abierta para preservar el bloqueo de lectura global).

Paso 2: Desde otra terminal de Linux, copiar físicamente los metadatos .cfg y el archivo de datos producto.ibd generados hacia la ruta de almacenamiento de backups segura:

```
cp /var/lib/mysql/practica_dml/producto.{ibd,cfg}
/backups/fisico_practica/
```

Paso 3: Una vez finalizada la copia física de los ficheros, liberar el bloqueo en la terminal origen:

```
UNLOCK TABLES;
```

Algoritmo de Restauración Física en el Servidor de Destino

Paso 4: En el servidor de destino (donde existe la tabla creada vacía), se debe descartar el espacio de tablas lógico existente para desvincular el archivo .ibd actual:

```
ALTER TABLE practica_dml.producto DISCARD TABLESPACE;
```

Paso 5: Copiar el archivo producto.ibd respaldado en el paso 2 dentro del directorio de la base de datos de destino /var/lib/mysql/practica_dml/ y asegurar que el propietario y grupo de permisos correspondan exactamente al daemon de MySQL:

```
cp /backups/fisico_practica/producto.ibd
/var/lib/mysql/practica_dml/
chown mysql:mysql /var/lib/mysql/practica_dml/producto.ibd
chmod 660 /var/lib/mysql/practica_dml/producto.ibd
```

Paso 6: Importar y vincular el nuevo archivo .ibd de datos en la estructura del diccionario del SGBD:

```
ALTER TABLE practica_dml.producto IMPORT TABLESPACE;
```

5.3 Copias Lógicas o Textuales (Dumping)

1. Sentencia SELECT ... INTO OUTFILE y LOAD DATA INFILE

Este método permite exportar rápidamente el contenido en bruto de una tabla a un fichero de texto plano formateado (por ejemplo, .csv), útil para migraciones rápidas de datos o análisis con herramientas como Excel.

Por motivos de seguridad contra ataques de inyección, MySQL restringe la exportación e importación de archivos mediante la variable del sistema **secure_file_priv**. Cualquier intento de exportación fuera del directorio definido en dicha variable lanzará el **Error 1290**.

```
-- Comprobar el directorio permitido para exportación
SHOW VARIABLES LIKE 'secure_file_priv';
-- En sistemas estándar, el directorio por defecto suele ser
'/var/lib/mysql-files/'
```

Exportación consistente de la tabla de empleados

```
SELECT codigo_empleado, nombre, apellido1, email, salario,
comision
INTO OUTFILE '/var/lib/mysql-files/empleados_export.txt'
FIELDS TERMINATED BY ','
OPTIONALLY ENCLOSED BY '"'
LINES TERMINATED BY '\n'
FROM practica_dml.empleado;
```

Restauración de los datos mediante LOAD DATA INFILE

Para importar los datos en una tabla de backup previamente estructurada:

```
USE practica_dml;
-- Crear tabla vacía de copia de seguridad (estructura clonada de
empleados)
CREATE TABLE empleado_backup LIKE empleado;
-- Importar el fichero de datos exportado
```

```
LOAD DATA INFILE '/var/lib/mysql-files/empleados_export.txt'
INTO TABLE empleado_backup
FIELDS TERMINATED BY ','
OPTIONALLY ENCLOSED BY '"'
LINES TERMINATED BY '\n';
```

2. La Utilidad mysqldump

Herramienta clásica y versátil que vuelca una base de datos o tablas en un archivo SQL con instrucciones DDL/DML.

A. Sintaxis para exportar tablas específicas de una base de datos:

```
mysqldump -u root -p practica_dml producto fabricante >
/backups/productos_fabricantes.sql
```

B. Sintaxis para exportar una o varias bases de datos completas:

```
# Exportar las bases de datos practica_dml y tienda de forma
estructurada
mysqldump -u root -p --databases practica_dml >
/backups/respaldo_practica.sql
```

(Nota: Al usar --databases, se insertan automáticamente las directivas CREATE DATABASE y USE en el fichero SQL de salida).

C. Sintaxis para exportar todas las bases de datos de la instancia de MySQL:

```
mysqldump -u root -p --all-databases >
/backups/instancia_completa.sql
```

Opciones Críticas de Producción para mysqldump

- --single-transaction: Esencial para tablas transaccionales **InnoDB**. Inicia una transacción antes de realizar la copia de seguridad, obteniendo un volcado consistente de los datos sin necesidad de bloquear los accesos de escritura a las aplicaciones cliente.

- --flush-logs (-F): Fuerza al servidor a inicializar un log binario nuevo justo al empezar el backup. Indispensable para organizar respaldos incrementales limpios.
- --no-data (-d): Exporta de forma exclusiva la estructura DDL de la base de datos (tablas, restricciones, tipos de datos), omitiendo los registros lógicos.
- ```
  --no-create-info (-t): Exporta únicamente los comandos INSERT INTO
  con los datos, omitiendo las instrucciones DDL de creación de
  tablas.
  ```
- ```
  --add-drop-table: Agrega la sentencia DROP TABLE IF EXISTS antes de
  cada definición de tabla. Evita errores de duplicados al restaurar bases
  de datos ya existentes.
  ```

🔗 **Documentación Oficial: Más Información en [MySQL Manual - mysqldump Utility]**

3. La Utilidad mysqlimport

Alternativa de línea de comandos de alto rendimiento al uso de LOAD DATA INFILE. Lee ficheros de texto formateados en csv y los inserta en la tabla correspondiente, la cual debe llamarse exactamente igual que el nombre base del archivo físico a importar.

```
# Importar el fichero 'empleado.txt' en la tabla 'empleado' del
esquema 'practica_dml'
mysqlimport -u root -p --fields-terminated-by=',' practica_dml
/var/lib/mysql-files/empleado.txt
```

6. ESCENARIO DE PRODUCCIÓN: PLAN DE RESPALDO INCREMENTAL Y RECUPERACIÓN POINT-IN-TIME

Caso Práctico: Respaldo Incremental de practica_dml en un Entorno Corporativo

Fase 1: Creación del Respaldo Base el Domingo a las 02:00 h

Como SysAdmin, ejecutamos un backup lógico consistente forzando la rotación de los logs binarios. El servidor comenzará a registrar cualquier cambio posterior en un nuevo binlog

secuencial (por ejemplo, binlog.000002).

```
mysqldump -u root -p --single-transaction --flush-logs --databases
practica_dml > /backups/practica_dml_domingo.sql
```

Fase 2: Actividad Normal e Incidencia el Lunes a las 10:00 h

Un desarrollador comete un fallo crítico de sintaxis en producción y ejecuta un DELETE accidental sin cláusula WHERE sobre la tabla de clientes en practica_dml, vaciando la tabla. El desastre se registra a las 10:00:15 en el log binario activo binlog.000002.

Fase 3: Proceso de Recuperación Point-in-Time (Lunes 10:15 h)

Paso 1: Forzar de manera inmediata un bloqueo de escritura para evitar que nuevos registros interfieran o cerrar las conexiones clientes y rotar los logs binarios para aislar la actividad actual:

```
mysqladmin -u root -p flush-logs
```

(Esto creará binlog.000003, cerrando binlog.000002 que contiene la sentencia destructiva).

Paso 2: Importar la copia de seguridad completa realizada el domingo, devolviendo la base de datos al estado consistente del domingo a las 02:00 h:

```
mysql -u root -p < /backups/practica_dml_domingo.sql
```

Paso 3: Analizar el log binario binlog.000002 de manera visual para encontrar la posición física o la fecha y hora exactas de la transacción dañina:

```
mysqlbinlog --database=practica_dml /var/lib/mysql/binlog.000002 |
grep -i -C 5 "delete"
```

Supongamos que encontramos que la transacción destructiva ocurrió a las "2026-06-22 10:00:15".

Paso 4: Aplicar de forma secuencial todas las transacciones realizadas desde el domingo hasta el instante previo a la eliminación accidental (10:00:14 h):

```
mysqlbinlog --database=practica_dml --stop-datetime="2026-06-22
10:00:14" /var/lib/mysql/binlog.000002 | mysql -u root -p
```

¡La base de datos se ha recuperado con total consistencia y sin pérdida de datos válidos!

7. HERRAMIENTAS GRÁFICAS Y MIGRACIÓN ENTRE SGBD

Realizar Copias de Seguridad con MySQL Workbench

1. En la barra lateral de administración, seleccionar **Data Export**.
2. Elegir las bases de datos y tablas que deseamos exportar.
3. Seleccionar entre exportar a un directorio de archivos individuales (para restauraciones selectivas más rápidas) o a un archivo consolidado continuo (Self-Contained File).
4. ```
   Activar las casillas Dump Stored Procedures and
   Functions, Dump Events y Dump
   Triggers según sea necesario para conservar la lógica de
   negocio de la unidad 11.
   ```
5. Pulsar **Start Export**.

Restauración mediante phpMyAdmin

1. Acceder al menú de navegación superior y pulsar sobre el botón **Importar** (Import).
2. Utilizar el explorador de archivos para seleccionar nuestro respaldo .sql o .sql.gz.
3. Definir el formato de lectura (estándar SQL).
4. Pulsar sobre el botón **Continuar** (Go).

Migración de Esquemas mediante MySQL Workbench Database Migration
   ```

MySQL Workbench incorpora un asistente integral para migrar datos desde orígenes heterogéneos (Microsoft SQL Server, Microsoft Access, PostgreSQL, Oracle o SQLite) hacia bases de datos compatibles con MySQL.

El proceso sigue esta jerarquía lógica de transformación de tipos y estructuras:

El asistente realiza la transformación lógica de la estructura de la base de datos de manera automatizada en los siguientes pasos:

1. **Establecimiento del Punto de Acceso (Connection Parameters):** Configura los controladores nativos (generalmente controladores ODBC o JDBC) para comunicarse con el servidor origen.
2. **Mapeo de Tipos de Datos (Reverse Engineering):** Analiza y adapta los tipos de datos incompatibles (por ejemplo, los campos de tipo IMAGE en SQL Server se traducen a campos de tipo LONGBLOB en MySQL, y los tipos de datos DATETIME2 se homologan al formato estándar DATETIME de precisión estándar).
3. **Alineación Física y Creación de Esquemas (Target Schema Creation):** Genera la base de datos de destino y crea las tablas aplicando las condiciones físicas óptimas.
4. **Carga Eficiente de Datos (Data Copy):** Ejecuta un volcado masivo de datos registro a registro de forma óptima deshabilitando temporalmente el control de claves ajenas (FOREIGN_KEY_CHECKS = 0) para acelerar el procesamiento.

8. DOCUMENTACIÓN DE POLÍTICAS DE SEGURIDAD Y CUMPLIMIENTO NORMATIVO (RGPD)

Un Administrador de Sistemas no puede limitarse a la ejecución de tareas técnicas; debe documentar de forma clara las normativas, roles y directivas que rigen la seguridad lógica y física de la información, de conformidad con el **Reglamento General de Protección de Datos (RGPD / LOPDGDD)**.

La política de seguridad física e informática de la empresa debe detallar explícitamente las siguientes pautas organizativas:

- **Identificación y Rotulación Estricta de Copias de Seguridad:** Cada dispositivo físico que almacene respaldos debe estar claramente catalogado con su descripción, fecha de

ejecución y clasificación de la información.

- **Establecimiento de Roles de Acceso (Mínimo Privilegio):** Despliegue de esquemas de seguridad basados en roles utilizando comandos DCL (GRANT/REVOKE). El personal no cualificado o externo no debe poseer privilegios globales de edición sobre esquemas sensibles.
- Gestión del Ciclo de Vida de los Respaldos (Conservación y Dest